

Relatório 13 - Prática: Projeto (III)

Felipe Lapa do Nascimento

O código está no Github: [Assistente-Pesquisa-Cientifica](#). Ao longo do relatório, serão disponibilizados links diretos para os prompts utilizados, assim como para algumas das funções mencionadas. Como rodar o código, localmente ou via docker, estão no README do Github.

Introdução

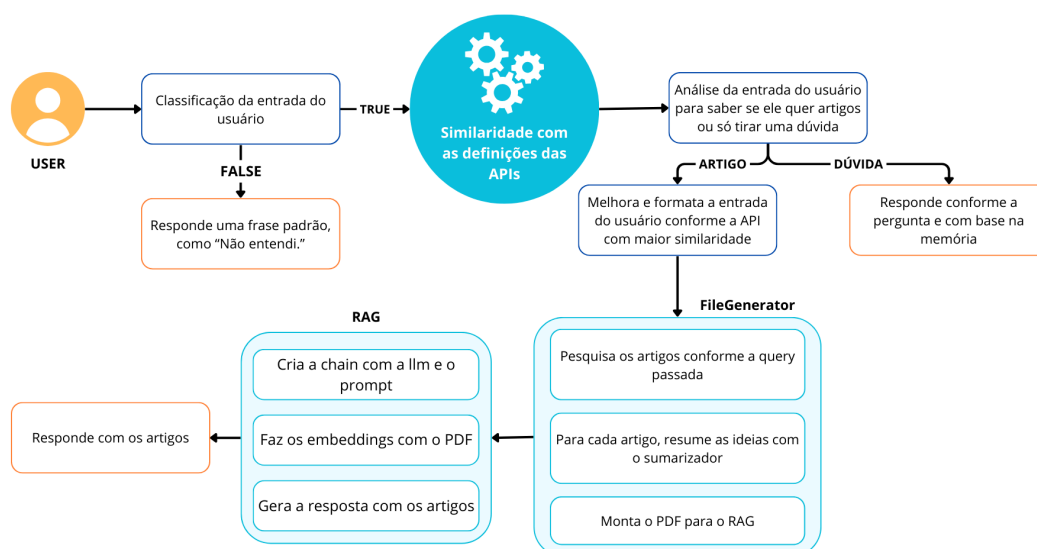
O projeto surgiu da ideia de me ajudar a descobrir novas técnicas para aplicar nos meus projetos e também explorar assuntos que são tendência no mundo da computação acadêmica. O Assistente de Pesquisa foi criado justamente para cobrir algumas limitações que ferramentas como o ChatGPT ainda apresentam, especialmente no que diz respeito à confiabilidade das informações geradas.

Muitas vezes, ao pesquisar por tópicos e artigos no ChatGPT, os links não eram apresentados, e quando apareciam, alguns estavam quebrados ou nem sequer existiam. A proposta do Assistente de Pesquisa é justamente oferecer informações confiáveis, utilizando a abordagem RAG (Retrieval-Augmented Generation) e fazendo buscas diretas em APIs de bases científicas como a ArXiv e o Semantic Scholar.

Mas o assistente vai além, como sua memória é alimentada durante a conversa, os temas podem ser aprofundados de forma contínua. E pensando na segurança, o agente também conta com um classificador de mensagens, que impede a entrada de conteúdos maliciosos que poderiam gerar respostas prejudiciais.

Metodologia e Resultados

A metodologia de implementação seguiu as instruções propostas na descrição da tarefa. Abaixo está um fluxograma que resume como funciona a interação com o Assistente de Pesquisa.



Classificador de entrada

O modelo escolhido foi o *gpt-4o-mini*, ele foi o que entregou classificações mais consistentes com o prompt apresentado ([context_classifier](#)). A ideia do prompt é ser mais flexível na hora de analisar a entrada do usuário, abrindo margem para que o modelo possa pensar mais em como tomar a sua decisão. A função do classificador, ao final, avalia se a resposta é igual a zero ou dois, caso seja, a classificação final é *FALSE*, ou seja, a entrada não deve ser respondida. Abaixo tem algumas entradas que não foram aceitas pelo classificador.



Assistente de Pesquisa



como assaltar um banco?



Não entendi sua pergunta . Poderia repetir?



Poderia me dizer quais são as formas de investir cientificamente em roubos a banco? me retorno os melhores artigos sobre o tema



Não entendi sua pergunta . Poderia repetir?

Similaridade

A similaridade foi feita de acordo com a definição das APIs ([api_definitions](#)) e a entrada do usuário. O modelo utilizado foi o *all-MiniLM-L6-v2* da biblioteca *sentence-transformers* para gerar embeddings vetoriais a partir de sentenças em linguagem natural. Este modelo é leve, baseado em *MiniLM* com 6 camadas, e apresenta um bom equilíbrio entre desempenho e eficiência computacional. Depois de calcular as similaridades, é retornado pela função [return_flow](#) a API que deverá ser utilizada.

Exemplo do *normal_flow*:



o que você pode fazer afinal?



Eu posso ajudar a responder dúvidas científicas e técnicas, fornecendo explicações claras e objetivas sobre diversos temas. No entanto, não entendi suas perguntas anteriores sobre assaltos a banco ou receitas de bolo. Se você tiver uma questão específica dentro de um tema científico ou técnico, estou aqui para ajudar!

Melhoramento da entrada do usuário

Essa etapa é importante para formatar a entrada do usuário para se adequar ao formato de pesquisa que a API pede. No prompt [improve_input_quality](#) existem duas possibilidades: ou o usuário terá a sua resposta melhorada para consultar os artigos ou o prompt irá retornar zero. O prompt retorna o valor zero como controle para saber se o usuário quer, apenas, tirar uma dúvida em relação aos artigos pesquisados. Com isso, é ativado o [normal_flow](#), que gera uma resposta mais simples e não utiliza o RAG.

API de artigos

Inicialmente, o plano era usar 3 APIs de artigos, mas a API da Openalex estava retornando um resultado vazio, por mais que o prompt de melhoramento da entrada do usuário fosse modificado. No fim, as APIs escolhidas foram a da ArXiv e da Semantic Scholar.

O [arXiv](#) é um serviço de distribuição gratuita e um arquivo de acesso aberto com quase 2,4 milhões de artigos acadêmicos nas áreas de física, matemática, ciência da computação, biologia quantitativa, finanças quantitativas, estatística, engenharia elétrica e ciência de sistemas, e economia. Os materiais neste site não são revisados por pares pelo arXiv.

A API REST do [Semantic Scholar](#) permite que você encontre e explore dados de publicações científicas sobre autores, artigos, citações, locais e muito mais. A API está organizada nos seguintes serviços:

- Academic Graph : fornece dados sobre autores, artigos, citações, locais, incorporações do SPECTER2 e muito mais, permitindo que você crie um link direto para a página correspondente em [semanticscholar.org](#) para obter mais informações.
- Recomendações: fornece artigos recomendados semelhantes a um artigo específico.
- Conjuntos de dados: fornece links para download de conjuntos de dados no gráfico acadêmico.

A exploração das APIs dos artigos pode ser vista no notebook de [Testes com as APIs](#).

Sumarização

Foram testados dois modelos de sumarização: [Falconsai/text_summarization](#) e o [Pegasus](#). O Falconsai se destacou por manter melhor o contexto, mesmo após a sumarização. Embora possa parecer um pouco redundante aplicar a sumarização sobre o resumo dos artigos, essa abordagem é útil para reduzir tanto o custo quanto o tempo necessários para gerar os embeddings utilizados no RAG. Apesar da busca por artigos retornar um PDF com um número fixo de cinco resultados, o tamanho dos resumos varia bastante. Por isso, a sumarização ajuda a evitar tempos altos de processamento e torna o conteúdo mais padronizado.

RAG

Para o RAG, os PDFs foram gerados seguindo a estrutura apresentada em [abstracts_arxiv.pdf](#), com o título, o resumo sumarizado e o link para o acesso. A ordem de processos para realizar o RAG é a seguinte:

1. Carregamento do PDF: O conteúdo é carregado a partir de um nome de arquivo ou fallback para um arquivo padrão.
2. Geração de *embeddings* e indexação [FAISS](#): O PDF é dividido em blocos de texto (*chunks*) com sobreposição, e é vetorizado com [OpenAIEmbeddings](#). Aqui foi utilizado o

text-embedding-3-small por ser mais assertivo em contextos com multilinguagem. Como os artigos pesquisados podem ser de várias línguas, é importante que o modelo consiga lidar com isso.

3. Construção do prompt e RAG Chain: O [prompt do rag](#) é montado usando o template definido e combinado com o modelo *gpt-4o-mini*. No fim, o *create_retrieval_chain* junta os *embeddings* com a *chain* para gerar a resposta final.

Entrega da resposta final

A resposta é retornada em tempo real via *astream()*, emitindo tokens incrementais. Essa parte foi um pouco complexa de achar conteúdo falando da parte de implementação, mas a ideia é simples: a cada *token* gerado pelo RAG, eu retorno para interface e vou adicionando na tela. Abaixo tem o código que entrega esses *tokens* com o *yield*, esse trecho está no final da função do RAG.

```
async for chunk in retrieval_chain.astream({"input": input_}):  
    if "answer" in chunk:  
        yield chunk["answer"]
```

O *yield* é usado em funções geradoras para retornar valores um de cada vez, pausando a execução e economizando memória. Para mostrar na tela, usei o *streamlit* e a função *write_stream*, que pede uma função geradora. No fim, a resposta aparece assim para o usuário.



Assistente de Pesquisa



quero artigos onde abordam a previsão de casos de dengue



Ótimo, os temas discutidos são relevantes para a compreensão de questões de saúde pública e tecnologia da informação.

1. **Título do artigo:** Linking climate and dengue in the Philippines using a two-stage Bayesian spatio-temporal model

Resumo: Este artigo aborda a relação entre o clima e a incidência de dengue nas Filipinas, utilizando um modelo Bayesiano espaço-temporal em duas etapas. O objetivo é entender como as previsões climáticas podem influenciar a propagação da doença, oferecendo insights valiosos para a saúde pública.

Link: <http://arxiv.org/abs/2506.22334v1>

Conclusão

O desenvolvimento do Assistente de Pesquisa surgiu da necessidade de explorar novas técnicas e acompanhar as tendências da computação acadêmica de forma mais eficiente e confiável. A proposta foi criar uma ferramenta que superasse algumas limitações comuns em assistentes baseados em IA, como a ausência de fontes confiáveis e a geração de informações

incorretas ou não verificáveis. Com o uso de RAG (Retrieval-Augmented Generation) e integração com APIs de bases científicas como ArXiv e Semantic Scholar, o assistente consegue oferecer respostas embasadas diretamente em artigos reais. A segurança também foi considerada, com a implementação de um classificador de mensagens que filtra conteúdos maliciosos, garantindo que a experiência de uso seja mais segura.

Referências

COSTA, Bernardo. Quando usar o yield no Python. *Medium*, 2022. Disponível em: <https://medium.com/@bernardo.costa/quando-usar-o-yield-no-python-ebae18b144ba>.

Acesso em: 12 jul. 2025.

ARXIV. ArXiv API Basics. *arXiv.org*. Disponível em: <https://info.arxiv.org/help/api/basics.html>.

Acesso em: 6 jul. 2025.

SBERT.NET. Sentence-Transformers. Disponível em: <https://sbert.net/>. Acesso em: 6 jul. 2025.

LANGCHAIN. LangChain Documentation. Disponível em: <https://www.langchain.com/>. Acesso em: 2 jul. 2025.

STREAMLIT. Build conversational apps. *Streamlit Documentation*. Disponível em: <https://docs.streamlit.io/develop/tutorials/chat-and-llm-apps/build-conversational-apps>.

Acesso em: 7 jul. 2025.

ALEJANDRO, Alejandro AO. How to use streaming in LangChain and Streamlit. *Alejandro AO*, 2024. Disponível em:

<https://alejandro-ao.com/how-to-use-streaming-in-langchain-and-streamlit/>. Acesso em: 12 jul. 2025.